PROGRAMACIÓN AVANZADA

Sesión 16

Apuntadores, Referencias y Arreglos

→ NOTAS DE CLASE

Universidad Panamericana 2026

Índice

1 La memoria del computador **2**
1.1 Tamaño de los tipos fundamentales 2

2 Apuntadores **3**
2.1 Declaración 3
2.2 Los dos operadores clave 3
2.3 Modificar el original a través del apuntador 4
2.4 Apuntador nulo 4

3 Referencias **4**
3.1 Diferencias: Apuntador vs Referencia 5

4 Arreglos **5**
4.1 Aritmética de apuntadores 5
4.2 Equivalencia fundamental 6

5 Comportamiento indefinido: acceso fuera de límites **6**

6 Relación arreglo / apuntador **7**
6.1 Array decay 7

7 Para saber más **7**
7.1 Lecturas recomendadas 8
7.2 Contexto histórico 8

8 Resumen **8**

La memoria del computador

Todo programa necesita guardar datos mientras se ejecuta. Esos datos viven en la **memoria RAM**, que puede imaginarse como un enorme arreglo de celdas numeradas. Cada celda tiene una dirección única (su número de casilla) y almacena exactamente un byte.

Dirección	Contenido	Variable
`0x61ff08`	16	`int x = 16;`
`0x61ff0c`	?	(próxima variable)

Cuando declaras `int x = 16;`, el compilador reserva 4 bytes contiguos en RAM, les asigna el valor 16 y les da el nombre `x`. El operador `&` (dirección-de) nos permite ver en cuál celda vive esa variable.

```
int x = 16;
cout << "Valor:     " << x  << endl;  // 16
cout << "Direccion: " << &x << endl;  // 0x61ff08  (varia
    cada ejecucion)
cout << "Tamano:    " << sizeof(x) << " bytes" << endl;  // 4
```

Listing 1: Direccion de una variable

Las direcciones se muestran en **hexadecimal** (base 16, prefijo `0x`). El valor exacto cambia en cada ejecución porque el sistema operativo decide dónde colocar las variables (*ASLR — Address Space Layout Randomization*).

• Tamaño de los tipos fundamentales

Tipo	Bytes (64-bit)	Rango típico
`char`	1	-128 a 127
`int`	4	$\approx \pm 2\,000\,000\,000$
`long`	8	$\approx \pm 9 \times 10^{18}$
`float`	4	$\approx \pm 3.4 \times 10^{38}$
`double`	8	$\approx \pm 1.7 \times 10^{308}$
`bool`	1	`true` / `false`
`pointer`	8	toda la memoria virtual del proceso

Arquitectura Von Neumann (1945): el principio de que los datos y el programa residen en la misma memoria es la base de todos los computadores modernos. Los apuntadores explotan directamente este modelo: son datos que apuntan a otros datos, o incluso al código mismo.

Referencia: J. von Neumann, “First Draft of a Report on the EDVAC”, 1945. Disponible en: `https://archive.org/details/firstdraftofrepo00vonn`

Apuntadores

Un **apuntador** es una variable cuyo valor es una *dirección de memoria*. En lugar de guardar un entero o un carácter, guarda *dónde* vive otro dato.

Variable normal	Apuntador

`+---------+`
`|  16  | <- int x`
`+---------+`
`0x61ff08`
	`+--------------+`
	`|  0x61ff08  | <- int* p`
	`+------+-------+`
	`|`
	`v`
	`+---------+`
	`|  16  | <- int x`
	`+---------+`

• Declaración

```
int* p;       // p es un apuntador a int
double* q;    // q es un apuntador a double
char* r;      // r es un apuntador a char
```

El tipo importa: el compilador necesita saber cuántos bytes leer al desreferenciar. Un `int*` lee 4 bytes; un `double*` lee 8.

• Los dos operadores clave

Operador	Nombre	Qué hace
`&x`	Dirección-de	Devuelve la dirección de memoria de `x`
`*p`	Desreferencia	Va a la dirección en `p` y lee/escribe el valor

El símbolo `*` tiene **dos significados** según el contexto:
En una declaración (`int* p;`): marca que `p` es un apuntador.
En una expresión (`*p`): desreferencia — lee el valor apuntado. No los confundas.

```
int  x = 16;
int* p;           // declaro el apuntador (aun no apunta a
    nada valido)

p = &x;           // p guarda la direccion de x

cout << p   << endl;  // direccion: 0x61ff08
cout << *p  << endl;  // valor:     16
cout << &p  << endl;  // p tambien vive en algun lugar de
    memoria
```

Listing 2: Apuntador: asignacion y desreferencia

• Modificar el original a través del apuntador

```
int  x = 16;
int* p = &x;

*p = 18;     // equivale exactamente a: x = 18;

cout << x;   // imprime 18
```

Listing 3: Escritura a traves de apuntador

✔ Recuerda

`*p = 18` y `x = 18` son instrucciones **idénticas** cuando `p = &x`. Ambas escriben el valor 18 en la misma celda de memoria.

• Apuntador nulo

Un apuntador que no apunta a nada válido debe inicializarse a `nullptr`. Desreferenciar un apuntador nulo produce un *segmentation fault* en tiempo de ejecución.

```
int* p = nullptr;    // apuntador nulo seguro

if (p != nullptr) {
    cout << *p;      // solo desreferenciamos si es seguro
}
```

Historia: En C el valor nulo era la macro `NULL` (definida como `0` o `(void*)0`). C++11 introdujo `nullptr`, un literal con tipo propio (`std::nullptr_t`), eliminando ambigüedades al sobrecargar funciones. Siempre usa `nullptr` en C++ moderno. *Referencia:* Stroustrup, B. (2013). *The C++ Programming Language*, 4.ª ed., §7.2.2.

Referencias

Una **referencia** es un *alias*: otro nombre para la misma variable. No ocupa espacio adicional en memoria — comparte exactamente la misma celda con la variable original.

```
int  x = 16;
int& y = x;     // y es alias de x

cout << &x << endl;  // 0x61ff08
cout << &y << endl;  // 0x61ff08  <- exactamente igual
```

Listing 4: Referencia: mismo espacio, otro nombre

La diferencia entre `&` en una **declaración** y en una **expresión**:
`int& y = x;` → declaración de referencia (alias).
`&x` en una expresión → operador dirección-de (devuelve la dirección).

- **Diferencias: Apuntador vs Referencia**

Característica	Apuntador (`int*`)	Referencia (`int&`)
¿Puede ser nulo?	Sí (`nullptr`)	No — debe inicializarse
¿Puede reasignarse?	Sí	No — fija tras la declaración
¿Ocupa memoria propia?	Sí (8 bytes en 64b)	No (alias)
Sintaxis para leer valor	`*p`	`r` (directo)
Uso típico	Memoria dinámica	Paso por referencia

¿Por qué existen las referencias? Bjarne Stroustrup las introdujo en C++ (ausentes en C) precisamente para soportar el paso por referencia en funciones con una sintaxis más segura y clara que los apuntadores crudos.
Referencia: Stroustrup, B. (1994). *The Design and Evolution of C++*, §3.7.

Arreglos

Un arreglo es un bloque de celdas **contiguas** en memoria, todas del mismo tipo. Para `int arr[5]`, el compilador reserva exactamente $5 \times 4 = 20$ bytes consecutivos:

`arr[0]`	`arr[1]`	`arr[2]`	`arr[3]`	`arr[4]`
10	20	30	40	50
0x100	0x104	0x108	0x10C	0x110

El nombre del arreglo es un **apuntador constante** al primer elemento: `arr` $\equiv$ `&arr[0]`.

- **Aritmética de apuntadores**

Sumar un entero a un apuntador no suma bytes: suma **múltiplos del tamaño del tipo**.

```
int arr[5] = {10, 20, 30, 40, 50};

cout << arr     << endl;  // 0x100  (direccion del primer
    elemento)
cout << arr + 1 << endl;  // 0x104  (+4 bytes, no +1)
cout << arr + 2 << endl;  // 0x108  (+8 bytes)

cout << *arr       << endl;  // 10  (arr[0])
cout << *(arr + 1) << endl;  // 20  (arr[1])
cout << *(arr + 2) << endl;  // 30  (arr[2])
```

Listing 5: Aritmetica de apuntadores

- **Equivalencia fundamental**

Para cualquier arreglo, las dos notaciones son **idénticas** en el código máquina generado:

$$\texttt{arr[i]} \equiv \texttt{*(arr + i)}$$

El compilador transforma internamente `arr[i]` en `*(arr + i)`. Los corchetes son azúcar sintáctica.

```
int arr[5] = {10, 20, 30, 40, 50};

for (int i = 0; i < 5; i++) {
    cout << arr[i]        // notacion clasica
         << " == "
         << *(arr + i)    // notacion con apuntador
         << endl;
}
```

Listing 6: Dos formas equivalentes de recorrer un arreglo

Dato curioso: dado que `arr[i]` $\equiv$ `*(arr + i)` y la suma es conmutativa, `arr[i]` $\equiv$ `i[arr]` es una expresión válida en C y C++. Es un truco de entrevistas técnicas — jamás lo uses en código real.

Comportamiento indefinido: acceso fuera de límites

C++ **no verifica** los límites de un arreglo en tiempo de ejecución. Acceder fuera es *undefined behavior* (UB): el resultado es impredecible.

```
int arr[5] = {10, 20, 30, 40, 50};

cout << arr[5];    // UB: lee memoria fuera del arreglo
cout << arr[-1];   // UB: idem hacia atras
```

El comportamiento indefinido no significa que el programa “fallará”. Puede imprimir basura, devolver datos de otra variable, no fallar en la máquina del desarrollador y sí en la del usuario, o permitir que un atacante controle el flujo del programa. **Es la fuente de muchas vulnerabilidades de seguridad críticas.**

El Gusano Morris (1988): el primer malware de gran escala en Internet explotó un *buffer overflow* (acceso fuera de límites en un arreglo) en el daemon `fingerd` de Unix. Infectó ∼6 000 máquinas, el 10 % del Internet de la época, y le costó a su autor la primera condena bajo la *Computer Fraud and Abuse Act*.
Referencia: Spafford, E. (1989). "The Internet Worm Program: An Analysis". *ACM SIGCOMM Computer Communication Review*, 19(1), 17–57. https://doi.org/10.1145/66093.66095

Para acceso seguro, usa `std::vector` con el método `.at(i)`, que lanza una excepción si el índice está fuera de rango:

```
#include <vector>
vector<int> v = {10, 20, 30, 40, 50};
cout << v.at(2);    // seguro: lanza std::out_of_range si i >=
    size
cout << v.at(9);    // lanza excepcion -- no UB
```

Relación arreglo / apuntador

- **Array decay**

Cuando usas el nombre de un arreglo en casi cualquier expresión, se convierte automáticamente en un apuntador al primer elemento. Esta conversión se llama ***array decay*** (decaimiento de arreglo).

```
int arr[5] = {10, 20, 30, 40, 50};
int* p = arr;    // equivale a: int* p = &arr[0];

// p y arr se comportan igual para indexar...
cout << p[2];    // 30  <- igual que arr[2]

// ...pero sizeof los distingue:
cout << sizeof(arr) << endl;  // 20  (5 elementos x 4 bytes)
cout << sizeof(p)   << endl;  //  8  (tamano de un apuntador
    en 64-bit)
```

✔ **Recuerda**

Consecuencia práctica del array decay: cuando pasas un arreglo a una función, la función solo recibe un apuntador; pierde la información del tamaño. Por eso las funciones que operan sobre arreglos siempre reciben el tamaño como parámetro adicional.

Para saber más

- ## Lecturas recomendadas
  - **cppreference — Pointer declaration:** https://en.cppreference.com/w/cpp/language/pointer
  - **cppreference — Reference declaration:** https://en.cppreference.com/w/cpp/language/reference
  - **Kernighan & Ritchie,** *The C Programming Language*, 2.ª ed. (1988), Capítulo 5: Pointers and Arrays. El capítulo que definió cómo se enseñan los apuntadores durante décadas.
  - **Stroustrup, B.** *A Tour of C++*, 3.ª ed. (2022), §1.7–1.8. Visión moderna y concisa de apuntadores en C++.
  - **C++ Core Guidelines — Bounds safety:** https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#S-bounds

- ## Contexto histórico

Los apuntadores llegaron a C de mano de **Dennis Ritchie** en los laboratorios Bell (1972), inspirados en el lenguaje B de Ken Thompson, que a su vez derivaba del BCPL de Martin Richards (1967). La motivación era práctica: escribir Unix de manera portable y eficiente sin ensamblador.

Ritchie recibió el Premio Turing en 1983 junto con Thompson "por el desarrollo del la teoría de los sistemas operativos genéricos, y en particular por la implementación del sistema operativo UNIX".

Referencia: Ritchie, D. M. (1993). "The Development of the C Language". *ACM SIGPLAN Notices*, 28(3), 201–208. https://dl.acm.org/doi/10.1145/154766.155580

Resumen

✔ **Recuerda**

Concepto	Símbolo	Qué hace
Dirección-de	`&x`	Dirección de memoria de `x`
Apuntador	`int* p`	Variable que guarda una dirección
Desreferencia	`*p`	Lee/escribe el valor en la dirección
Referencia	`int& r = x`	Alias: mismo espacio, otro nombre
Array decay	`arr` en expr.	Se convierte en `&arr[0]`
Aritmética	`p + i`	Avanza `i*sizeof(tipo)` bytes
Equivalencia	`arr[i]`	Idéntico a `*(arr+i)`